Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнила:
Студент 3 курса
Группы ИИ-26
Карпович К.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     - **А.** С фиксированным шагом обучения α
     - **Б.** С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях α
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Допускается применение математических и графических библиотек (NumPy, Matplotlib). Использование ML-фреймворков (PyTorch, Keras и т.д.) запрещено.

Вариант 6

№ of variant	n	Task
6	6	AND

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

N_INPUTS    = 6
LOGIC_FUNC  = "AND"
C           = 1.0        # ширина сигмоидной функции
ETA_FIXED   = 0.1
MAX_EPOCHS  = 5000
ERROR_GOAL  = 0.01
THRESHOLD   = 0.5        # порог округления выхода → класс 0/1
TRAIN_RATIO = 0.75       # доля обучающей выборки
```

```
# ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
def sigmoid(s, c=C):
    return 1.0 / (1.0 + np.exp(-c * s))


def generate_truth_table(n: int, func: str) -> tuple[np.ndarray, np.ndarray]:
    all_inputs = list(itertools.product([0, 1], repeat=n))
    X = np.array(all_inputs, dtype=float)  # (64, 6)

    if func == "AND":
        y = np.array([int(all(row)) for row in all_inputs], dtype=float)
    elif func == "OR":
        y = np.array([int(any(row)) for row in all_inputs], dtype=float)
    elif func == "XOR":
        y = np.array([int(sum(row) % 2 == 1) for row in all_inputs], dtype=float)
    elif func == "NAND":
        y = np.array([int(not all(row)) for row in all_inputs], dtype=float)
    elif func == "NOR":
        y = np.array([int(not any(row)) for row in all_inputs], dtype=float)
    else:
        raise ValueError(f"Неизвестная функция: {func}")

    return X, y


def train_test_split_manual(X, y, train_ratio=TRAIN_RATIO, seed=42):
    np.random.seed(seed)
    train_idx, test_idx = [], []

    for cls in np.unique(y):
        cls_indices = np.where(y == cls)[0]
        np.random.shuffle(cls_indices)
        n_train = max(1, int(len(cls_indices) * train_ratio))
        train_idx.extend(cls_indices[:n_train])
        test_idx.extend(cls_indices[n_train:])

    train_idx = np.array(train_idx)
    test_idx  = np.array(test_idx)
    return X[train_idx], y[train_idx], X[test_idx], y[test_idx]


def compute_total_error(X, y, weights, bias):
    total = 0.0
    for xi, ei in zip(X, y):
        s  = np.dot(weights, xi) - bias   # S = W·X - T
        yi = sigmoid(s)
        total += 0.5 * (yi - ei) ** 2
    return total


def predict(X, weights, bias):

    s = X @ weights - bias
    probs = sigmoid(s)
    classes = (probs >= THRESHOLD).astype(int)
    return probs, classes


def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred) * 100.0

# АЛГОРИТМ ОБУЧЕНИЯ
def train_perceptron(X_train, y_train, X_test, y_test,
                     adaptive=False, eta=ETA_FIXED,
                     max_epochs=MAX_EPOCHS, error_goal=ERROR_GOAL,
                     seed=0):
    np.random.seed(seed)
    n = X_train.shape[1]

    # Инициализация весов и порога в малом диапазоне [0, 1]
    weights = np.random.uniform(0, 0.5, size=n)
    bias    = np.random.uniform(0, 0.5)

    train_errors = []
    test_errors  = []

    for epoch in range(1, max_epochs + 1):
        # Адаптивный шаг: η = 1/p
```

```
        eta_t = 1.0 / epoch if adaptive else eta

        for xi, ei in zip(X_train, y_train):
            s  = np.dot(weights, xi) - bias
            yi = sigmoid(s)
            delta = yi - ei

            weights -= eta_t * delta * xi
            bias    += eta_t * delta

        # Вычисление суммарной ошибки после эпохи
        E_train = compute_total_error(X_train, y_train, weights, bias)
        E_test  = compute_total_error(X_test,  y_test,  weights, bias)
        train_errors.append(E_train)
        test_errors.append(E_test)

        if E_train < error_goal:
            print(f"  Сходимость достигнута на эпохе {epoch}   "
                  f"(E_train = {E_train:.6f})")
            break
    else:
        print(f"  Максимум эпох ({max_epochs}) исчерпан. "
              f"E_train = {train_errors[-1]:.6f}")

    return weights, bias, train_errors, test_errors, epoch


def main():
    print("=" * 65)
    print(f" n = {N_INPUTS} входов | Функция: {LOGIC_FUNC}")
    print("=" * 65)

    #Генерация таблицы истинности
    X_all, y_all = generate_truth_table(N_INPUTS, LOGIC_FUNC)
    total_samples = len(y_all)
    n_ones  = int(y_all.sum())
    n_zeros = total_samples - n_ones

    print(f"\n[1] Таблица истинности для {LOGIC_FUNC} с {N_INPUTS} входами:")
    print(f"    Всего наборов : {total_samples}  (2^{N_INPUTS})")
    print(f"    Класс '1'     : {n_ones}  набор(ов)")
    print(f"    Класс '0'     : {n_zeros} набор(ов)")
    print(f"\n    Единственный набор с выходом 1:")
    print(f"    x = {X_all[y_all == 1][0].astype(int)}  →  y = 1")

    # Разбиение на обучающую / тестовую выборки
    X_train, y_train, X_test, y_test = train_test_split_manual(
        X_all, y_all, train_ratio=TRAIN_RATIO
    )
    print(f"\n[2] Разбиение выборки:")
    print(f"    Обучающая : {len(y_train)} наборов  "
          f"({len(y_train)/total_samples*100:.0f}%)")
    print(f"    Тестовая  : {len(y_test)}  наборов  "
          f"({len(y_test)/total_samples*100:.0f}%)")

    # Эксперимент А: фиксированный шаг обучения
    print(f"\n[3] ═══ Эксперимент А: Фиксированный шаг η = {ETA_FIXED} ═══")
    w_fix, b_fix, err_train_fix, err_test_fix, ep_fix = train_perceptron(
        X_train, y_train, X_test, y_test,
        adaptive=False, eta=ETA_FIXED
    )
    prob_train_fix, cls_train_fix = predict(X_train, w_fix, b_fix)
    prob_test_fix,  cls_test_fix  = predict(X_test,  w_fix, b_fix)
    acc_train_fix = accuracy(y_train, cls_train_fix)
    acc_test_fix  = accuracy(y_test,  cls_test_fix)

    print(f"\n  Итоговые веса    : {np.round(w_fix, 5)}")
    print(f"  Итоговый порог T : {b_fix:.5f}")
    print(f"  Эпох обучения    : {ep_fix}")
    print(f"  Точность (обуч.) : {acc_train_fix:.1f}%")
    print(f"  Точность (тест.) : {acc_test_fix:.1f}%")

    # Эксперимент Б: адаптивный шаг η = 1/p
    print(f"\n[4] ═══ Эксперимент Б: Адаптивный шаг η = 1/p ═══")
    w_ada, b_ada, err_train_ada, err_test_ada, ep_ada = train_perceptron(
        X_train, y_train, X_test, y_test,
        adaptive=True
    )
```

```
    prob_train_ada, cls_train_ada = predict(X_train, w_ada, b_ada)
    prob_test_ada,  cls_test_ada  = predict(X_test,  w_ada, b_ada)
    acc_train_ada = accuracy(y_train, cls_train_ada)
    acc_test_ada  = accuracy(y_test,  cls_test_ada)

    print(f"\n  Итоговые веса    : {np.round(w_ada, 5)}")
    print(f"  Итоговый порог T : {b_ada:.5f}")
    print(f"  Эпох обучения    : {ep_ada}")
    print(f"  Точность (обуч.) : {acc_train_ada:.1f}%")
    print(f"  Точность (тест.) : {acc_test_ada:.1f}%")

    print("\n[5] Построение графиков...")
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    fig.suptitle(
        f"Однослойный персептрон | {LOGIC_FUNC}-{N_INPUTS} ",
        fontsize=14, fontweight='bold'
    )

    for ax, (err_tr, err_te, ep, title) in zip(
        axes,
        [
            (err_train_fix, err_test_fix, ep_fix,
             f"А. Фиксированный шаг η = {ETA_FIXED}"),
            (err_train_ada, err_test_ada, ep_ada,
             "Б. Адаптивный шаг η = 1/p"),
        ]
    ):
        epochs = np.arange(1, len(err_tr) + 1)
        ax.plot(epochs, err_tr, label="Обучающая выборка", color="royalblue", lw=2)
        ax.plot(epochs, err_te, label="Тестовая выборка",  color="tomato",    lw=2,
                linestyle="--")
        ax.axhline(ERROR_GOAL, color="gray", linestyle=":", lw=1.2,
                   label=f"Цель E = {ERROR_GOAL}")
        ax.set_title(title, fontsize=12)
        ax.set_xlabel("Эпоха")
        ax.set_ylabel("Суммарная ошибка $E_s$")
        ax.legend()
        ax.grid(True, alpha=0.4)
        ax.set_xlim(1, max(len(err_tr), 10))
        ax.set_ylim(bottom=0)

    plt.tight_layout()
    plt.savefig("convergence_plot.png", dpi=150)
    plt.show()
    print("  График сохранён: convergence_plot.png")

    # Оценка обобщающей способности
    print("\n[6] Оценка обобщающей способности (тестовая выборка):")
    print(f"    {'x':>30}  {'e':>4}  {'p(1)':>8}  {'ŷ':>4}  {'OK':>4}")
    print("    " + "-" * 55)
    n_correct = 0
    for xi, ei, pi, yi in zip(X_test, y_test, prob_test_fix, cls_test_fix):
        ok = "□" if yi == int(ei) else "□"
        if yi == int(ei):
            n_correct += 1
        row = " ".join(str(int(v)) for v in xi)
        print(f"    [{row}]  {int(ei):>4}  {pi:>8.4f}  {yi:>4}  {ok:>4}")
    print(f"\n    Правильно: {n_correct}/{len(y_test)} = {n_correct/len(y_test)*100:.1f}%")

    # Сравнительная таблица экспериментов
    print("\n[7] Сравнение экспериментов:")
    print(f"  {'Параметр':<35} {'Фиксированный':>15} {'Адаптивный':>15}")
    print("  " + "-" * 68)
    print(f"  {'Шаг обучения':<35} {str(ETA_FIXED):>15} {'1/p':>15}")
    print(f"  {'Эпох до сходимости':<35} {ep_fix:>15} {ep_ada:>15}")
    print(f"  {'E_train (финал)':<35} {err_train_fix[-1]:>15.6f} {err_train_ada[-1]:>15.6f}")
    print(f"  {'E_test  (финал)':<35} {err_test_fix[-1]:>15.6f} {err_test_ada[-1]:>15.6f}")
    print(f"  {'Точность обуч., %':<35} {acc_train_fix:>15.1f} {acc_train_ada:>15.1f}")
    print(f"  {'Точность тест., %':<35} {acc_test_fix:>15.1f} {acc_test_ada:>15.1f}")

    # Режим функционирования
    print("\n" + "=" * 65)
    print("  РЕЖИМ ФУНКЦИОНИРОВАНИЯ")
    print("  (используются веса из Эксперимента А — фиксированный шаг)")
    print("=" * 65)
    inference_mode(w_fix, b_fix)

# РЕЖИМ ФУНКЦИОНИРОВАНИЯ (INFERENCE)
```

```
def inference_mode(weights, bias):
    print(f"\nВведите {N_INPUTS} бит через пробел (например: 1 1 1 1 1 1)")
    print("Для выхода введите 'q'\n")

    while True:
        raw = input(f"  Вход ({N_INPUTS} бит) > ").strip()
        if raw.lower() == 'q':
            print("Выход из режима функционирования.")
            break
        try:
            bits = list(map(int, raw.split()))
            if len(bits) != N_INPUTS:
                print(f"  □ Нужно ровно {N_INPUTS} значений. Попробуйте снова.")
                continue
            if not all(b in (0, 1) for b in bits):
                print("  □ Значения должны быть 0 или 1.")
                continue
            x = np.array(bits, dtype=float)
            s = np.dot(weights, x) - bias
            prob = sigmoid(s)
            cls  = int(prob >= THRESHOLD)
            print(f"  Взвешенная сумма S = {s:.4f}")
            print(f"  Вероятность P(y=1) = {prob:.6f}")
            print(f"  Класс              = {cls}  "
                  f"({'□ AND=1' if cls == 1 else '□ AND=0'})\n")
        except ValueError:
            print("  □ Ошибка ввода. Введите целые числа 0 или 1.\n")

if __name__ == "__main__":
    main()
```

Результат программы:

```
C:\Users\Ксения\PycharmProjects\PolyclinicARM\.venv\.venv\Scripts\pytho
================================================================
 n = 6 входов | Функция: AND
================================================================

[1] Таблица истинности для AND с 6 входами:
    Всего наборов : 64  (2^6)
    Класс '1'     : 1  набор(ов)
    Класс '0'     : 63 набор(ов)

    Единственный набор с выходом 1:
    x = [1 1 1 1 1 1]  →  y = 1

[2] Разбиение выборки:
    Обучающая : 48 наборов  (75%)
    Тестовая  : 16  наборов  (25%)

[3] ═══ Эксперимент А: Фиксированный шаг n = 0.1 ═══
  Сходимость достигнута на эпохе 3157  (E_train = 0.009996)

  Итоговые веса    : [5.47734 5.45178 5.4865  5.46204 5.5186  1.64768]
  Итоговый порог T : 27.10726
  Эпох обучения    : 3157
  Точность (обуч.) : 100.0%
  Точность (тест.) : 93.8%

[4] ═══ Эксперимент Б: Адаптивный шаг n = 1/p ═══
  Максимум эпох (5000) исчерпан. E_train = 0.378102
```

```
  Итоговые веса    : [0.78751 0.82445 0.43321 0.93036 1.23928 0.49431]
  Итоговый порог T : 6.33576
  Эпох обучения    : 5000
  Точность (обуч.) : 97.9%
  Точность (тест.) : 100.0%

[5] Построение графиков...
  График сохранён: convergence_plot.png

[6] Оценка обобщающей способности (тестовая выборка):
                                x     e      p(1)     ŷ    OK
    -------------------------------------------------------
    [1 1 0 0 1 0]     0    0.0000     0     ✓
    [1 0 0 1 1 1]     0    0.0001     0     ✓
    [1 0 0 0 1 1]     0    0.0000     0     ✓
    [0 1 0 1 1 1]     0    0.0001     0     ✓
    [1 1 0 1 0 0]     0    0.0000     0     ✓
    [0 0 1 0 1 0]     0    0.0000     0     ✓
    [0 1 0 1 1 0]     0    0.0000     0     ✓
    [0 1 0 0 1 0]     0    0.0000     0     ✓
    [1 1 1 1 1 0]     0    0.5718     1     ×
    [0 1 0 1 0 0]     0    0.0000     0     ✓
    [0 0 0 1 1 1]     0    0.0000     0     ✓
    [1 0 1 0 1 0]     0    0.0000     0     ✓
    [0 0 1 1 1 0]     0    0.0000     0     ✓
    [0 1 1 1 0 0]     0    0.0000     0     ✓
    [1 1 0 0 1 1]     0    0.0001     0     ✓
    [1 0 0 1 1 0]     0    0.0000     0     ✓
```

```
    Правильно: 15/16 = 93.8%

[7] Сравнение экспериментов:
  Параметр                                  Фиксированный      Адаптивный
  ------------------------------------------------------------------------
  Шаг обучения                                        0.1            1/p
  Эпох до сходимости                                 3157           5000
  E_train (финал)                                0.009996       0.378102
  E_test  (финал)                                0.163451       0.012925
  Точность обуч., %                                 100.0           97.9
  Точность тест., %                                  93.8          100.0

==================================================================
  РЕЖИМ ФУНКЦИОНИРОВАНИЯ
  (используются веса из Эксперимента А — фиксированный шаг)
==================================================================

Введите 6 бит через пробел (например: 1 1 1 1 1 1)
Для выхода введите 'q'
```

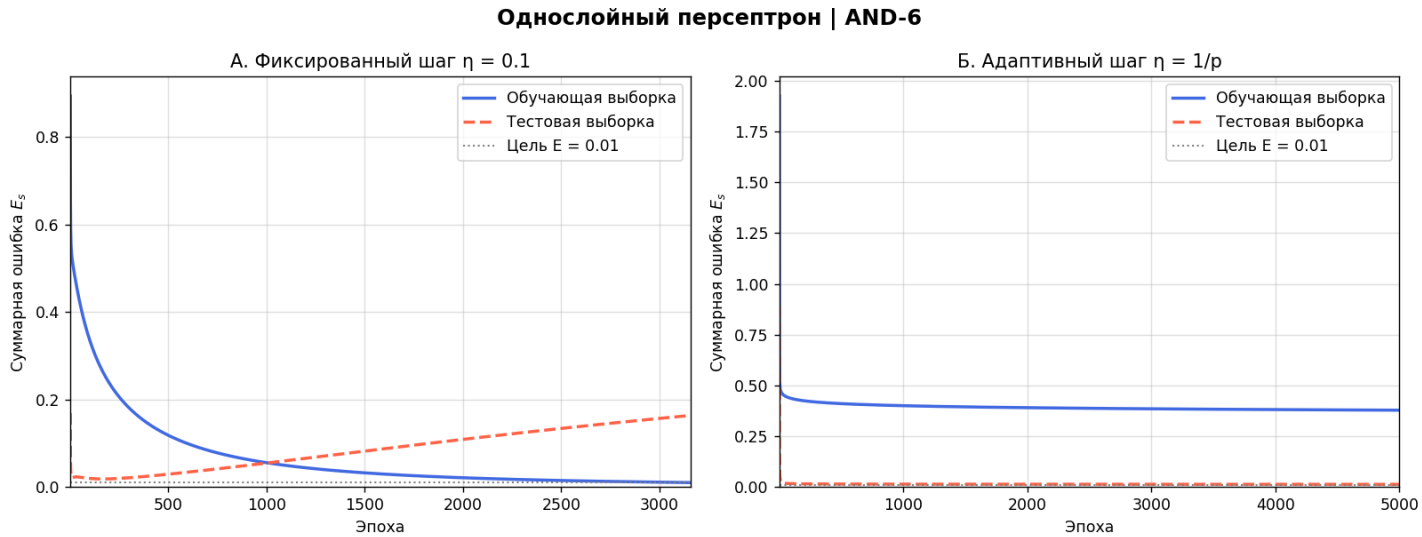
```
  Вход (6 бит) > 1 1 1 1 1 1
  Взвешенная сумма S = 1.9367
  Вероятность P(y=1) = 0.873988
  Класс              = 1  (✓ AND=1)

  Вход (6 бит) > 1 1 1 1 1 0
  Взвешенная сумма S = 0.2890
  Вероятность P(y=1) = 0.571754
  Класс              = 1  (✓ AND=1)

  Вход (6 бит) > 0 0 0 0 0 0
  Взвешенная сумма S = -27.1073
  Вероятность P(y=1) = 0.000000
  Класс              = 0  (× AND=0)

  Вход (6 бит) > q
Выход из режима функционирования.

Process finished with exit code 0
```



Вывод: в ходе работы был спроектирован и реализован однослойный сигмоидальный персептрон для синтеза логической функции AND от 6 входов. Проделанная работа подтвердила, что однослойные нейронные сети способны эффективно решать задачи классификации линейно разделимых образов и восстанавливать сложные логические закономерности по ограниченной выборке.